

Configurations multi-environnement (MSPR3 / TPRE601)

Trois configurations de base de la stack HealthAI Coach, plus des overlays additionnels (exposition publique, classification photo, deploiement continu, acceleration GPU), toutes orchestrees via Docker Compose. Elles se combinent avec un compose de base : `docker-compose.yml` (mode prod, images GHCR) ou `docker-compose.dev.yml` (mode dev, build local).

1. Complete

Tous les services actifs, IA generative operationnelle, monitoring complet.

```
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml up -d
```

- API, Auth, ETL, AI-Nutrition (+ Ollama), Reco-Fitness, Front, PostgreSQL, MongoDB.
- Observabilite complete (voir `monitoring/README.md`).
- IA : Ollama local par default ; Mistral possible si `MISTRAL_API_KEY` est renseignee.

2. Offline

Mode degrade demonstrable sans connexion internet.

```
docker compose -f docker-compose.yml -f docker-compose.offline.yml up -d
```

- LLM force sur Ollama local (aucun appel Mistral).
- Auth sans verification email (aucun appel Resend) : inscription et connexion fonctionnent hors-ligne (variable `AUTH_OFFLINE=true`).
- Front branche sur le backend local (donnees seedees par l'ETL).
- Prerequis (une seule fois, avec internet) : images construites/pull et modele Ollama gemma3:4b telecharge. Ensuite, tout tourne hors-ligne.

3. Performance

Optimisee pour materiel modeste : limites CPU/RAM, monitoring simplifie.

```
# Coeur de stack avec limites de ressources, sans IA generative lourde
docker compose -f docker-compose.yml -f docker-compose.performance.yml up -d \
    db auth-db auth-migrate auth api etl mongo reco-fitness front

# Monitoring simplifie (Prometheus + Grafana + cAdvisor)
docker compose -f docker-compose.monitoring.yml up -d prometheus grafana cadvisor
```

- Limites CPU/RAM par service (`docker-compose.performance.yml`).
- IA generative (`ai-nutrition` + `ollama`) omise du démarrage par défaut : la generation de plans repas retombe sur la matrice statique de secours. Pour l'inclure malgre tout, ajouter `ai-nutrition ollama` a la liste de services.
- Monitoring reduit aux metriques essentielles (conteneurs + Prometheus + Grafana).

Overlays additionnels

Ces overlays se combinent par-dessus une configuration de base (typiquement "complete").

Exposition publique (Traefik)

`docker-compose.traefik.yml` place la plateforme derriere le reverse proxy Traefik, avec TLS. Front, API, Auth, AI-Nutrition, Reco-Fitness et Grafana sont routes sur des sous-domaines `*.zespri.duckdns.org` (certresolver `duckdns` , certificat wildcard). Les variables CORS et les URLs publiques sont alignees sur l'origine du front. Necessite le reseau externe `proxy_net` .

```
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml \
    -f docker-compose.traefik.yml up -d
```

Classification photo (vision / Mistral)

`docker-compose.vision.yml` bascule `ai-nutrition` sur le backend `mistral_vision` (`ANALYZE_BACKEND=mistral_vision`) pour la classification d'aliments par photo (route A1), avec repli automatique sur Food-101 en cas d'echec. Utilise l'image officielle GHCR de `ai-nutrition` .

Deploiement continu (Watchtower)

`docker-compose.watchtower.yml` ajoute Watchtower : il interroge GHCR toutes les 5 minutes et redeploie automatiquement les services backend des qu'une nouvelle image `:latest` est publiee (detail dans `CICD.md`). Le front (build local) est exclu.

Acceleration GPU (Ollama)

`docker-compose.gpu.yml` reserve le GPU NVIDIA de l'hote pour le conteneur Ollama et garde le modele charge en VRAM (`OLLAMA_KEEP_ALIVE=-1` , une inference a la fois pour tenir sur les cartes a 4 Go). Prerequis : driver NVIDIA et NVIDIA Container Toolkit sur l'hote. Le passage sur GPU rend la

generation Gemma3:4b exploitable (le run CPU documente 200 000+ ms par plan, au-dela du timeout applicatif) et permet d'activer le prompt few-shot complet (`FEW_SHOT_FULL_EXAMPLES=true` cote ai-nutrition) ainsi que l'eval LLM n=30 (depot AI-Nutrition, `docs/GPU_EVAL_PLAYBOOK.md`).

```
docker compose -f docker-compose.yml -f docker-compose.gpu.yml up -d
```

Combinaison de production (serveur)

Sur le serveur, les overlays sont combines pour une plateforme exposee, supervisee et maintenue a jour automatiquement :

```
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml \  
-f docker-compose.traefik.yml -f docker-compose.vision.yml \  
-f docker-compose.watchtower.yml up -d
```

Une fois le passthrough GPU valide sur l'hote (toolkit NVIDIA configure), ajouter `-f docker-compose.gpu.yml` a cette combinaison.

Sauvegarde, restauration, remise a zero

Voir `scripts/backup.sh`, `scripts/restore.sh`, `scripts/clean.sh` (section dediee du `README.md`).